



# Guía para conseguir el 100% en el "Hito 3"

El Hito 3 marca la consolidación de la arquitectura interna de sus proyectos backend en Java puro. El objetivo de esta etapa es reestructurar el repositorio abandonando los diseños monolíticos o acoplados, aplicando **Arquitectura Limpia** y patrones tácticos de **Diseño Guiado por el Dominio (DDD)** mediante el patrón Repositorio como frontera tecnológica.

Para alcanzar el puntaje máximo (**10/10 - Nivel "Excelente"**), su repositorio de backend Java debe cumplir estrictamente con tres pilares evaluables. Aquí les explicamos exactamente qué buscará el evaluador y cómo lograrlo.

## Pilar 1: Arquitectura de Carpetas y Capas Limpias (3 Puntos)

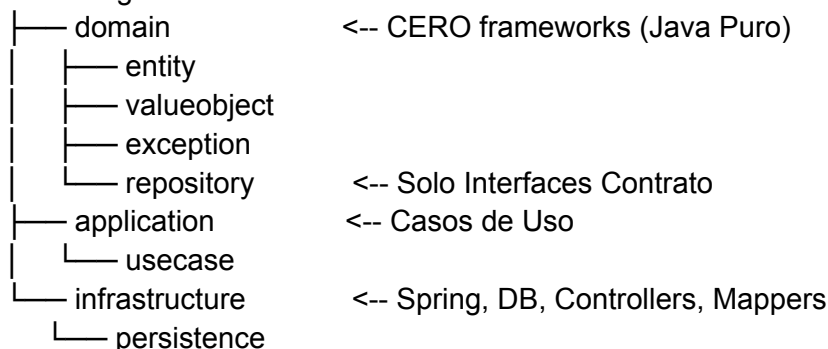
**El objetivo:** Reorganizar los paquetes del proyecto separando netamente las capas de Dominio, Aplicación e Infraestructura, garantizando que el núcleo del negocio permanezca 100% inmune a acoplamientos tecnológicos.

### Checklist para el Nivel "Excelente":

- **Segmentación Estricta de Paquetes:** La estructura del proyecto debe reflejar los tres paquetes base explícitos: **domain**, **application** e **infrastructure**.
- **Cumplimiento de la Regla de Dependencia Estricta:** Las capas externas pueden importar e invocar a las internas, pero las internas jamás deben conocer ni referenciar a las externas.
- **Cero Acoplamiento Tecnológico en el Núcleo:** Está estrictamente prohibido incluir importaciones o anotaciones de frameworks externos (como Spring **@Service**, **@Repository**, Jackson o JPA **@Entity**) dentro del paquete **domain** o **application**. El Dominio debe ser Java puro.

Estructura de Paquetes Esperada:

com.tunegocio



## Pilar 2: Modelado de Patrones Tácticos de DDD (3 Puntos)

**El objetivo:** Construir el modelo de negocio utilizando clases ricas en comportamiento, identificando Entidades con ciclo de vida e inmutabilidad en los Objetos de Valor.

### Checklist para el Nivel "Excelente":

- **Entidades con Identidad Única:** Modelar las clases principales con un identificador unívoco (*id*) que trascienda los cambios de sus atributos internos.
- **Value Objects (Record) Auto-Validantes:** Sustituir los tipos primitivos sueltos (*String*, *double*) por Objetos de Valor inmutables desarrollados preferentemente con *record* de Java. Sus constructores deben aplicar validación defensiva para rechazar datos corruptos al momento de la instanciación.
- **Encapsulamiento del Dominio:** Exponer métodos públicos con nombres descriptivos alineados al negocio en lugar de puros *getters/setters* anémicos.

Java

```
// src/main/java/com/tunegocio/domain/valueobject/Email.java
package com.tunegocio.domain.valueobject;
```

```
import com.tunegocio.domain.exception.InvalidEmailException;
```

```
// 1. Inmutabilidad nativa con 'record'
```

```
public record Email(String value) {
```

```
    // 2. Constructor compacto para auto-validación defensiva
```

```
    public Email {
```

```
        if (value == null || value.isBlank() || !value.contains("@")) {
```

```
            throw new InvalidEmailException("Formato de correo electrónico inválido.");
```

```
        }
```

```
        value = value.trim().toLowerCase();
```

```
    }
```

```
}
```

Java

```
// src/main/java/com/tunegocio/domain/entity/Customer.java
package com.tunegocio.domain.entity;
```

```
import com.tunegocio.domain.valueobject.Email;
```

```
// 1. Entidad con Identidad Única (Sin anotaciones de JPA/Spring)
```

```
public class Customer {
```

```
    private final String id;
```

```
    private String name;
```

```
private Email email; // Uso de Value Object para proteger el formato

public Customer(String id, String name, Email email) {
    if (id == null || id.isBlank()) {
        throw new IllegalArgumentException("El ID del cliente no puede ser nulo.");
    }
    this.id = id;
    this.name = name;
    this.email = email;
}

public String getId() { return id; }
public String getName() { return name; }
public Email getEmail() { return email; }
}
```

## Pilar 3: Desacoplamiento por Contratos de Repositorios (4 Puntos)

**El objetivo:** Diseñar el almacenamiento a través de interfaces abstractas en el dominio, asegurando que los Casos de Uso dependan únicamente de estas abstracciones mediante inyección por constructor.

### Checklist para el Nivel "Excelente":

- **Interfaces de Repositorio Puras:** Definir las interfaces de los repositorios en el paquete `domain.repository` escritas en lenguaje omnipresente. Deben operar exclusivamente con objetos de dominio y no deben heredar de interfaces de frameworks (como `JpaRepository`) en esta capa.
- **Casos de Uso Cohesivos:** Crear clases enfocadas en un único flujo de negocio en `application.usecase` que reciban la interfaz del repositorio como parámetro.
- **Inyección por Constructor OBLIGATORIA:** Queda prohibido instanciar la implementación concreta de un repositorio dentro del Caso de Uso utilizando la palabra clave `new` (`new PostgresRepository()`). La dependencia debe exigirse en el constructor.

Java

```
// src/main/java/com/tunegocio/domain/repository/BookingRepository.java
package com.tunegocio.domain.repository;
```

```
import com.tunegocio.domain.entity.Booking;
import java.util.Optional;
```

```
// 1. Contrato abstracto en el Dominio (Sin dependencias tecnológicas)
public interface BookingRepository {
    void save(Booking booking);
    Optional<Booking> findById(String id);
}
```

Java

```
// src/main/java/com/tunegocio/application/usecase/CreateBookingUseCase.java
package com.tunegocio.application.usecase;
```

```
import com.tunegocio.domain.entity.Booking;
import com.tunegocio.domain.repository.BookingRepository;
```

```
// 1. Caso de Uso aislado que orquesta el negocio
public class CreateBookingUseCase {
    private final BookingRepository bookingRepository; // Dependencia de la interfaz
```

```
// 2. Inyección obligatoria por constructor (Cero uso de 'new')
public CreateBookingUseCase(BookingRepository bookingRepository) {
    this.bookingRepository = bookingRepository;
}

public void execute(Booking newBooking) {
    if (bookingRepository.findById(newBooking.getId()).isPresent()) {
        throw new IllegalStateException("La reserva ya se encuentra registrada.");
    }
    this.bookingRepository.save(newBooking);
}
}
```



## El Entregable: Un README.md Profesional

Su repositorio actualizado en GitHub debe incluir un archivo `README.md` claro en la raíz con:

1. **Título y Descripción:** Nombre del proyecto integrador autónomo y explicación de la solución reestructurada bajo Arquitectura Limpia.
2. **Diagrama o Mapa de Paquetes:** Breve mapa del proyecto explicando la separación entre `domain`, `application` e `infrastructure`.
3. **Instrucciones de Ejecución de Pruebas:**
  - Para compilar y verificar el proyecto: `mvn clean compile`
  - Para ejecutar la suite de pruebas unitarias que validan el desacoplamiento: `mvn test`